

飞桨静态图新执行器设计文档

目录

- 一、背景介绍
- 二、设计实现
 - （一）静态预处理阶段
 - 1. pass优化
 - 2. kernel选择
 - 3. 数据转换
 - 4. 依赖分析与建立
 - 5. 线程调度分析
 - 6. 变量生命周期分析
 - 7. 多流分析
 - （二）动态调度阶段
 - 1. OP调度
 - 2. 多流同步
 - 3. 显存GC

一、背景介绍

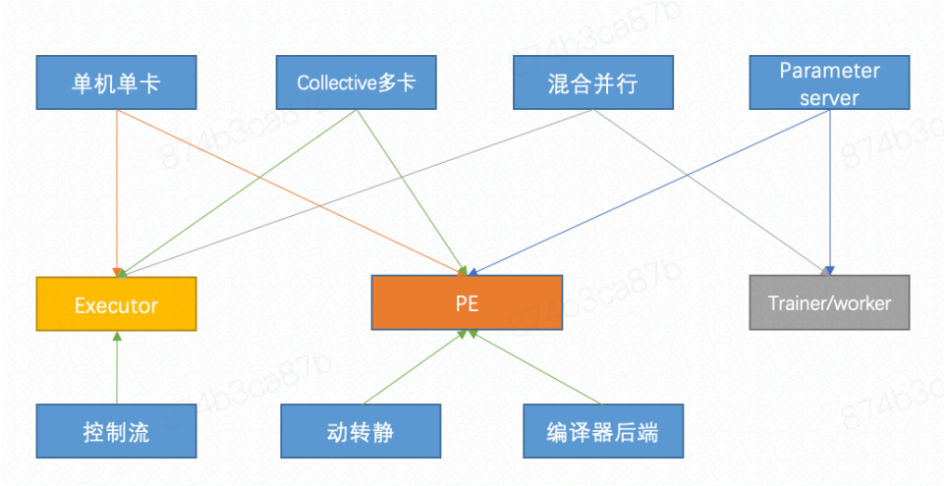
在深度学习框架中，执行器作为计算图的调度执行引擎，主要负责管理计算图中算子（OP）执行所需的资源，并将OP按照合理的顺序调度执行，保证较高的资源利用率和OP调度性能。

由于历史原因，飞桨框架中存在Executor、ParallelExecutor (PE)、Traniner/Worker等多套执行器，以适应不同的业务场景和需求。其中，

（1）Executor是一个OP-by-OP的执行器，会将Program中的每个OP顺序执行，其执行方式简单直接、调试方便，但不包含图优化、显存inplace、OP并发执行等机制，导致在部分任务上性能表现不佳；

（2）PE是面向SSA Graph的执行器，其中针对不同的业务场景具有多套不同的SSAGraphExecutor实现，支持多卡、图优化、高效显存管理、多线程并发等机制，但PE动态性较强导致runtime overhead较高、不支持多流和异构设备，且代码实现复杂、各个子模块耦合度高，不利于功能升级和维护；

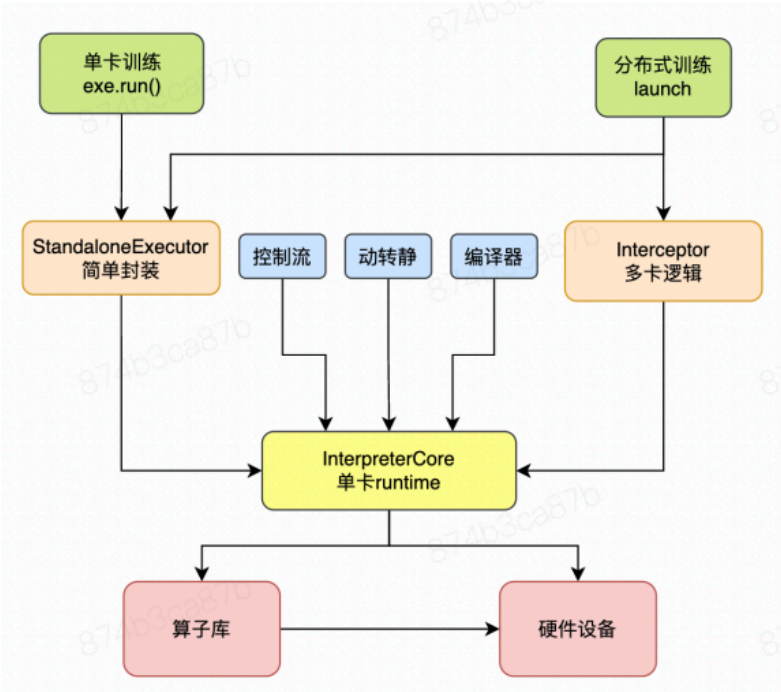
（3）Trainer/Worker则是分布式场景下为了支持Parameter Server和Pipeline模型并行而使用的执行方式。



飞桨历史上针对不同业务场景和需求使用不同的执行器

为了解决同时存在多套执行器所带来的**代码碎片化严重、用户理解和使用门槛过高、框架维护和迭代困难、上下游模块接入成本过大**等问题，飞桨在2.3版本重新设计和开发了一套全新的静态图执行器（下文简称新执行器）。

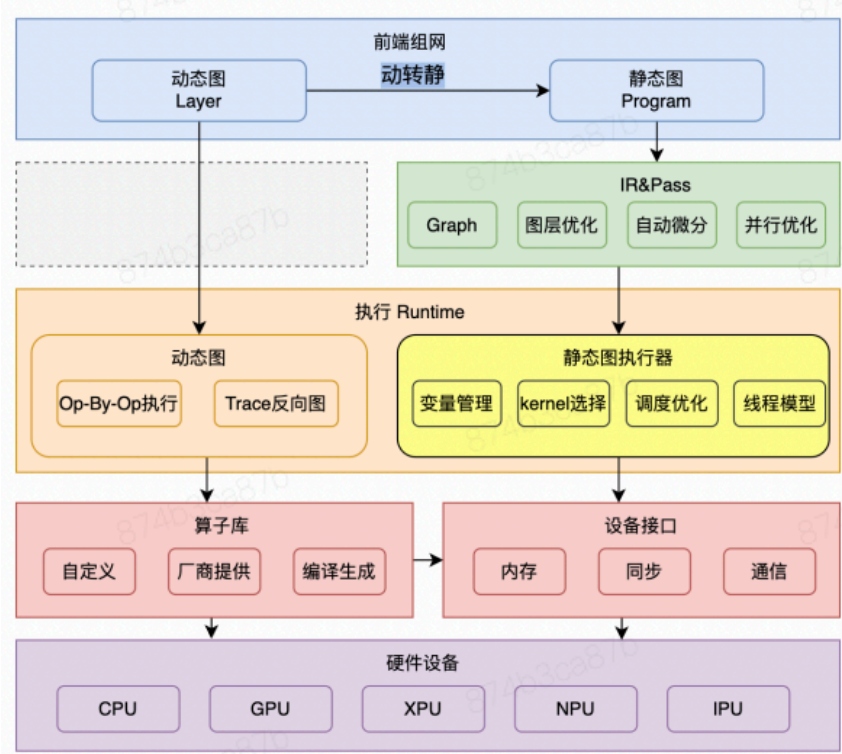
新执行器项目旨在通过打造一套**多场景通用、性能全面领先、功能易于扩展**的静态图执行器，来统一现有的多套执行器，使得框架中所有场景均使用同一个执行器，以减少后续新功能的开发维护以及上下游模块的接入成本。即包括单卡训练、分布式训练等Python端入口，以及控制流、动转静、编译器、C++端使用到执行器的场景都调用新执行器的核心runtime（InterpreterCore），如下图所示。



新执行器功能定位

统一后的新执行器，向上与模型组网和IR/Pass等静态图API交互、承接静态图模式下各种场景的模型运行，向下与算子库、硬件设备、显存分配器（Allocator）、数据加载器（Dataloader）等框架各个组件交互、与各个组件紧密配合保证模型的高效训练，同时也作为框架动转静、控制

流、CINN编译器等有算子调度场景的后端执行引擎。当前，新执行器已较为成熟稳定，在各种不同场景下具有比其它旧执行器更优的性能表现。

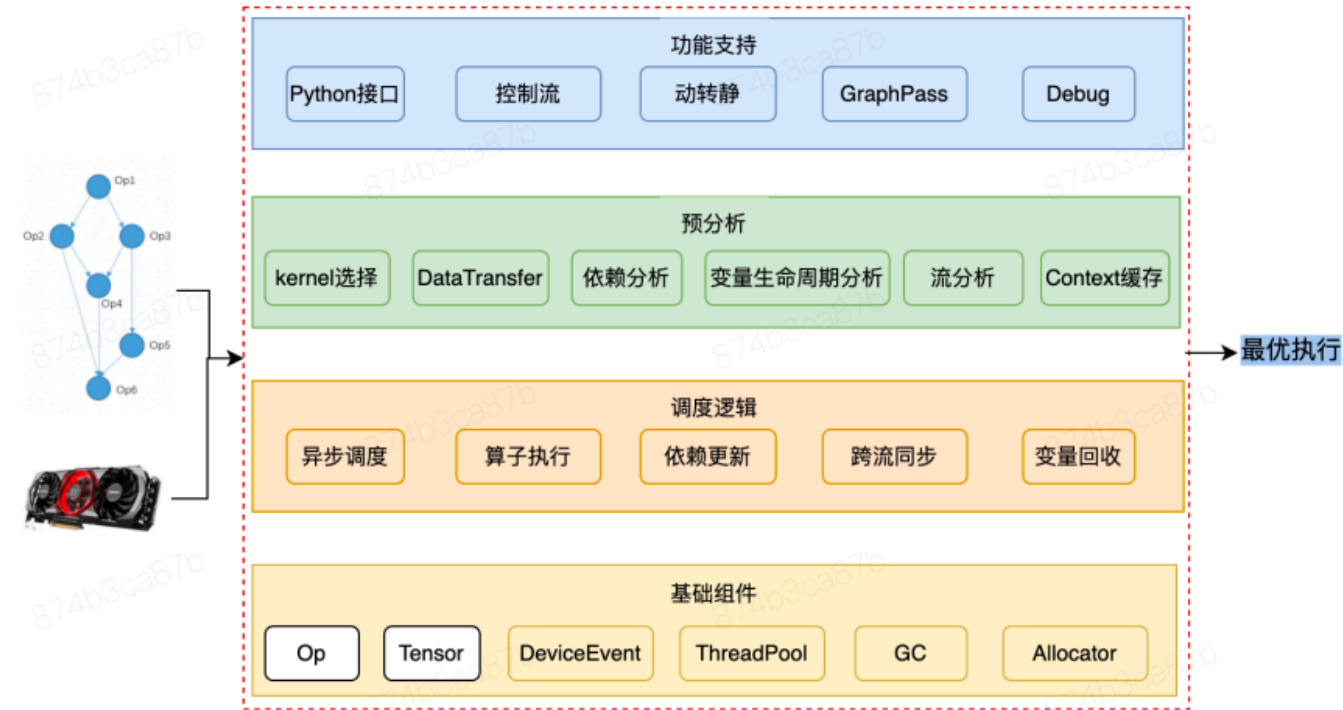


静态图执行器在框架中所处的位置

本文主要介绍新执行器中一些核心模块和关键技术的设计和实现。

二、设计实现

全新的静态图执行器整体架构将分为四层，如下图所示：



执行器总体架构

最上层是面向各种应用场景而提供的调用接口，最底层是为了OP高效执行而实现的各种高性能基础组件。而执行调度的核心逻辑集中在中间两层，执行器会根据给定的计算图和硬件资源编排OP调度顺序、映射硬件资源，以得到最优的执行方式，并按照最优的方式调度执行。中间两层实际对应了执行器执行模型的两个工作阶段：**静态预处理和动态调度**。

整体来说，在新执行器的设计理念中，我们希望把尽可能多的工作放到静态预分析中完成，这样就能减少动态调度时候的开销。

（一）静态预处理阶段

静态预处理阶段包括运算资源的管理和初始化以及计算图的预分析和优化两部分。

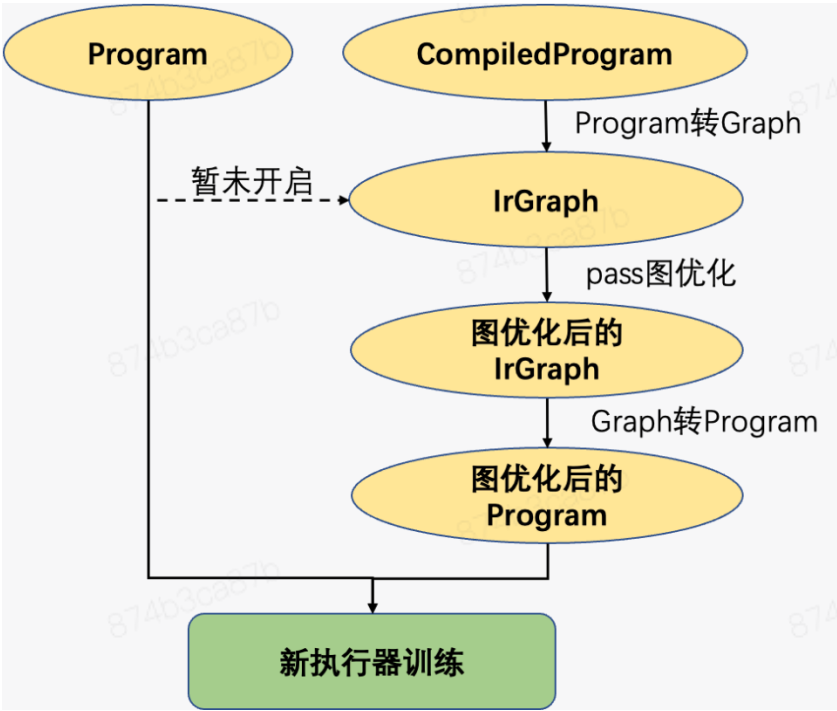
运算资源包括OP和变量的实例，也包括OP执行所需的系统资源，如线程池、多流异步执行所需的stream和event、分布式通信资源、eigen和cudnn handle等。这些资源大多根据资源特征及算子计算的实际需求实现了精细化管理，引入诸如线程池延迟创建、线程数自适应选择、stream按需初始化、event复用等机制，以最大程度地提高资源利用率。

针对计算图的预分析，则包括pass优化、kernel选择、数据转换、依赖分析、线程调度分析、变量生命周期分析、多流分析等多项技术。

1. pass优化

历史上，飞桨框架中存在大量基于Graph IR实现的图优化pass，利用算子融合、梯度合并、显存inplace等计算图层级的优化手段来提升模型的训练性能。为了复用这些图优化能力，新执行器中实现并优化了Program和Graph这两种模型表示IR互转的技术，对于用户组好的Program，在静态预处理阶段会首先被转换成Graph，在Graph上利用pass做计算图优化，然后再将优化后的计算图回转成Program进行训练。

由于Executor和PE两套执行器对应的用户前端接口也不同，Executor主要用于执行Program，PE主要用于执行CompiledProgram，因而在旧执行器中，只有执行CompiledProgram才会应用图优化技术。当前为了与旧行为对齐，新执行器也只对CompiledProgram前端应用了pass优化，Program前端则暂未开启。后续执行器统一后，两种前端也将相应地进行统一，并都默认开启图优化技术。



新执行器中的图优化

2. kernel选择

针对不同的后端硬件、数据类型、计算库等，一个OP往往有多个不同的计算kernel实现。在给定计算图、硬件设备和输入信息后，一个OP需要执行的计算kernel就是确定的。

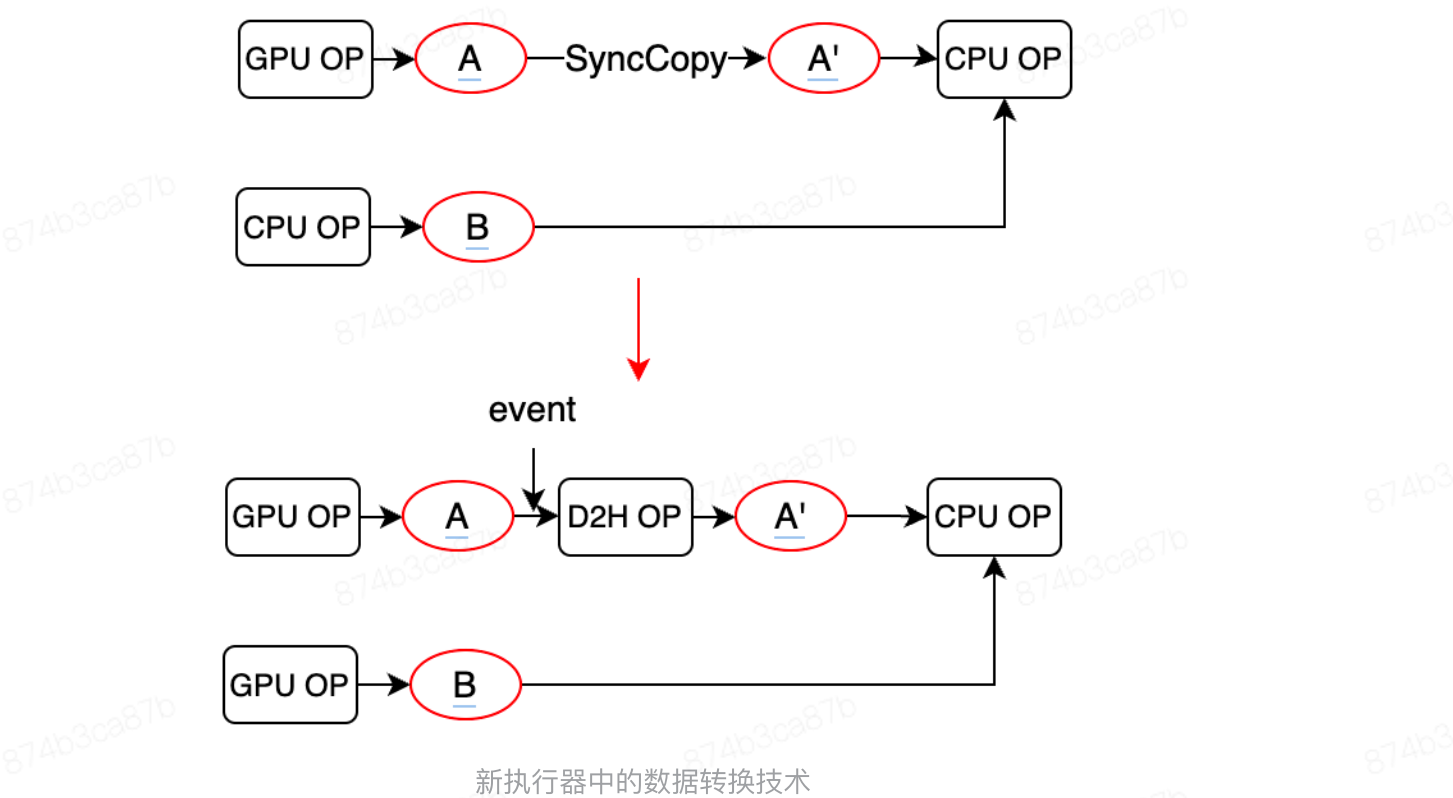
但在飞桨旧的OP体系设计下，算子的kernel需要基于运行时信息来做选择，是一个动态的过程。在预处理阶段，新执行器采用“一轮动态调度+缓存”的方式实现kernel选择的“半静态化”，通过实际执行一轮算子来实现kernel的动态选择，并将动态选择的结果缓存下来供后续的训练轮次使用，以减少后续训练中kernel选择的开销。

当前，飞桨大部分算子已迁移至新的phi体系，新执行器针对kernel选择“完全静态化”的工作正在进行中。

3. 数据转换

当一个OP的输入数据类型、存储位置或内存布局与该OP的执行kernel所需类型不对应时，则需要在该kernel执行前对输入数据进行转换。在旧执行器中，数据转换一般在OP执行阶段动态地进行判断并调用转换操作。而新执行器预处理阶段在做完kernel选择后，会提前分析出计算图中需要插入数据转换的位置，并在相应的位置插入数据转换OP，使得在后续的执行过程中都不再需要额外进行数据转换的考虑。

通过插入OP的形式进行数据转换，将数据转换操作作为OP调度的“一等公民”，在设计上实现了操作的一致性、简化了调度逻辑、且使得数据转换可以和其它计算操作一同参与规划，实现全局最优的调度。



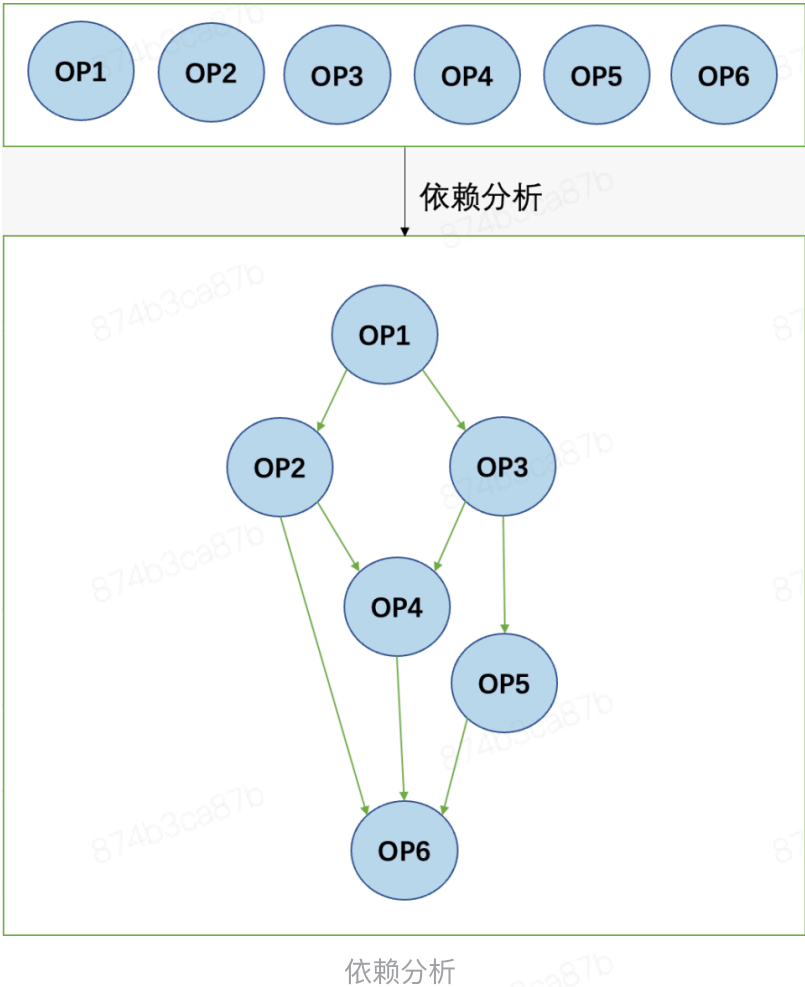
新执行器中的数据转换技术

4. 依赖分析与建立

依赖分析本质上是Program IR中的OP列表重新表达成一张并发友好的计算图。由于Program IR中将模型表示成一个OP列表，其语义隐含了列表中的OP会被逐个按顺序调度执行的假设，而顺序执行在很大程度上限制了通过多线程并发提高调度性能的可能。为了支持并发调度，新执行器在预处理阶段会对模型中的依赖情况进行细粒度地分析，重新建立算子间的依赖关系，只保留下必须的依赖关系，去除Program IR中算子按顺序调度执行的假设下所存在的一些不合理的隐式依赖，从而为后续的算子实际调度争取更大的并发可能。

当前新执行器中建立依赖主要考虑以下几种情况：

- (1) 数据依赖：op1->A->op2，数据A作为op1的输出，同时作为op2的输入
- (2) 控制依赖：上层显式通过控制边表明op1和op2之间存在依赖关系，当前主要通过depend_op实现
- (3) 特定属性的算子需要严格按Program中的顺序执行：如通信算子、随机数算子等
- (4) 有inplace行为的算子：如coalesce_tensor_op等
- (5) 其它一些需要特殊考虑的算子：如read_op等



5. 线程调度分析

为了方便调度，新执行器中将OP划分成同步和异步两种类型。同步OP表示该OP的所有运行逻辑均是同步的，即该OP或不存在device端操作（如CPU OP），或虽存在device端操作但操作是同步的，kernel在host端执行完成即可保证device端也执行完成（如同步GPU OP）；异步算子则表示该OP的运行逻辑中存在异步的device操作。

OP类型用于指导线程池的分发，同步OP和异步OP会被分发到不同的线程池中进行调度。由于device操作的拉起往往是互斥的（如CUDA launch kernel是有锁的），因而异步OP对应的线程池中只有一个线程，所有异步OP实际只会串行执行。而对于同步OP，其对应线程池中的线程数量会根据系统实际硬件资源的情况来决定，常见的线程数量为4个。

在静态预处理阶段，每个OP都会提前分析出其调度完成后，下游的OP是否需要分发到别的线程中去执行。每个OP都会记录其下游有哪些OP是需要分发到别的线程执行的，哪些是需要保持在当前线程执行的。对于同步OP来说，其下游OP中只会保留一个同步OP在当前线程中执行，其余OP均会根据其同步/异步类型重新进行线程分发；而对于异步OP来说，其下游的所有异步OP均会保留在当前线程执行，所有同步OP都会分发到其它线程执行。



为了指导显存垃圾回收 (garbage collection, 简称GC), 新执行器在预处理阶段会提前分析出每个变量的最后存活OP列表 (即计算图中最后需要使用到该变量的OP)。后续实际调度阶段, 在每个OP执行完成之后, 均会尝试对不再需要的变量进行显存回收, 以降低模型训练的显存开销。

如下图所示例子，变量X同时需要被OP2和OP3使用，OP3是变量X的最后存活OP，则OP2执行之后变量X的显存还不能回收，需要在OP3执行之后再尝试对X进行显存回收。最后存活OP的预处理，既可以保证模型训练过程中临时变量的及时回收，又能保证对显存GC的尝试操作尽可能地少。



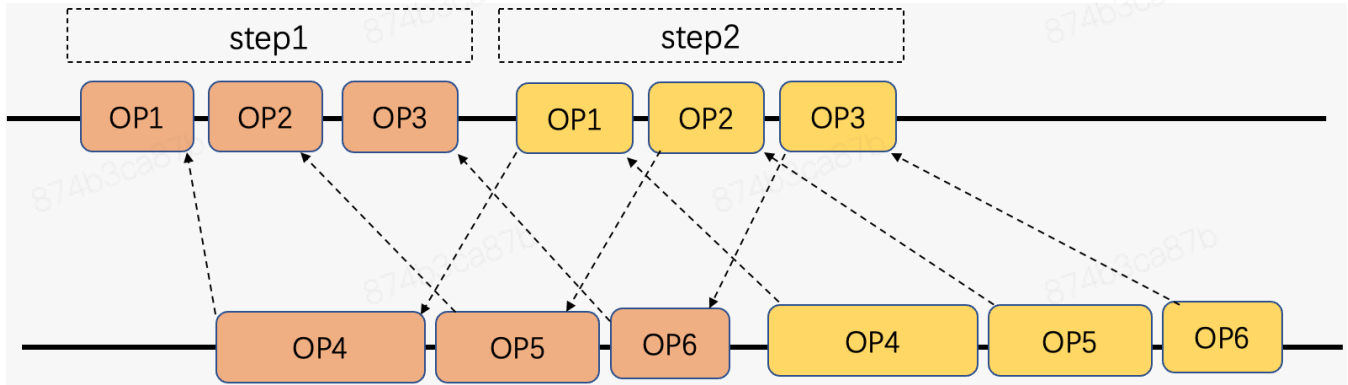
为了尽可能提高硬件资源的利用率，新执行器中引入了多流调度，以让计算、拷贝、通信等依赖不同硬件资源的操作可以尽可能地重叠执行。在静态预分析阶段，需要针对多流调度的场景提前分配每个OP的执行流，并进行多流之间同步的分析和规划。

在OP执行流分配上，执行器底层设计了计算流、H2D拷贝流、D2H拷贝流和通信流4种不同类型的流，大多数计算OP的操作均会分配到计算流上进行，数据拷贝相关OP的操作则会被分配到H2D和D2H拷贝流中进行，而一些通信相关的OP会被分配到通信流上进行。此外，为了满足分布式场景下对新执行器多流能力的二次开发需求，新执行器还支持在Program IR中通过OP属性灵活指定每个OP的执行流。

预处理阶段为所有OP创建并分配执行流之后，为了保证多流异步执行场景下计算结果的正确性，还需要针对被跨流读写的变量进行同步规划。同步规划主要是为了保证有数据依赖的上下游OP在不同的流中仍能按照依赖顺序执行，以防止多流下OP实际执行顺序错乱导致运算结果出错。在实际执行阶段，多流同步会通过设备提供的event事件机制来进行。对于需要跨流同步的两个上下游OP，通过在上游OP调度完成后插入event记录事件，在下游OP调度完成之前等待对应的event完成，来达到同步的目的。

在静态预处理阶段分析和规划event事件的插入和等待时机时，对于每个下游OP，会在与其具有数据依赖关系且不属于同一个流的上游OP中绑定一个需要记录的event事件，并将该event绑定到该下游OP需要等待的事件列表中。若某个流中存在多个上游OP需要与同一个下游OP进行跨流同步，则下游OP只会与该流中最后一个被调度执行的上游OP做同步操作。若存在多个下游OP需要与同一个上游OP进行同步，则用于同步的event会进行复用。

除了同一轮训练里的OP需要进行多流同步之外，跨两轮训练之间的OP也可进行多流重叠，新执行器对跨两轮训练之间的OP也同样进行了多流同步的考虑和规划，以保证两轮训练之间多流重叠异步执行不会产生错误。



新执行器中的跨训练轮次多流重叠

(二) 动态调度阶段

新执行器在静态预处理阶段完成了大量的分析和准备工作，这些分析和准备会被缓存下来，后续的每一轮执行都不需要再重复进行，因而到实际执行的动态调度阶段是非常轻量级和高效率的。在动态调度阶段，核心的逻辑主要包括OP异步调度、多流同步和显存GC三部分。

1. OP调度

OP调度本质上是让预处理阶段建立好的计算图按合适的拓扑序执行。在实际执行计算图时，新执行器具有两种执行模式，一种是多线程的异步调度模式，一种是单线程的trace调度模式。

对于多线程异步调度模式，执行器中会动态维护每个OP的引用计数，以记录该OP还有多少个上游OP未调度完成。初始时引用计数为0的OP即为计算图中没有前向依赖的OP，这些OP会首先被根据其同步/异步类型分发到对应的线程池中调度执行。当一个OP调度完成后，会对其所有下游OP的引用计数减1，并触发对引用计数被减到0的下游OP的调度。由于预处理阶段已经提前分

析好了下游OP被触发调度时哪些需要保持在当前线程执行、哪些需要分发到其它线程执行，因而对被触发的下游OP的多线程调度是简单且高效的。

除了预先做好线程规划外，为了更高效地进行多线程异步调度，新执行器还基于eigen的nonblocking-threadpool开源代码实现了一套**高性能线程池**，并针对飞桨模型训练的需求和特点进行了较多的细粒度优化。新执行器实现的高性能线程池组件具有以下优势：

(1) 提升线程调度性能

- 降低addTask的入队时延：避免线程休眠、避免锁碰撞等
- 降低task在队列里的排队时延：使用local队列、支持负载均衡机制、支持work stealing等

(2) 提升线程池本身性能

- 避免cache false sharing：强制数据在不同cacheline进行对齐
- 避免锁竞争：尽可能无锁化，必要时使用细粒度锁，使用自旋锁而不是互斥锁等
- 让渡cpu：在spinlock中加入cpuRelax,释放cpu

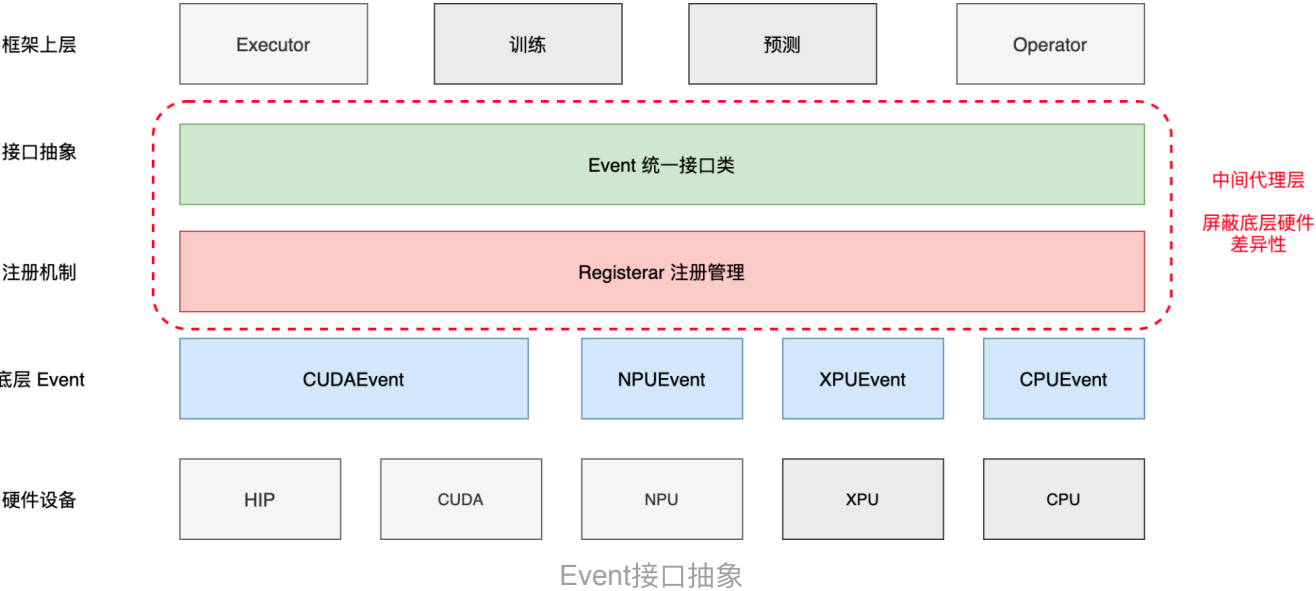
(3) 支持追踪task，与主线程通过消息队列通信，提前结束等功能

另一种调度OP的trace模式则主要应用于模型中不存在同步OP的场景。在这种场景下，所有OP都是串行执行的异步OP，不需要额外创建线程池进行异步调度，只需要规划好合适的OP拓扑序、然后通过for-loop的方式调度OP即可。单线程调度的trace模式可完全抹去特定场景下多线程创建和切换的开销，当前主要被用于动转静中。

2. 多流同步

由于在静态预处理阶段已经提前分析和规划好了OP之间的同步关系，并为上下游OP分别绑定了需要记录和等待的event事件，因而在动态调度阶段，新执行器只需要在每个上游OP调度完成之后通过硬件接口插入event记录事件，并在每个下游OP调度执行之前等待对应的event事件完成即可。

由于不同的硬件平台对event机制具有不同的实现方式和接口形式，且针对下游算子不同的同步/异步类型，也需要不同的等待机制（同步类型算子等待需要阻塞host端CPU线程，异步类型算子等待只需要在device端执行流上插入异步等待事件），因而新执行器中通过设计统一的event接口类以及对应的Registrar注册管理机制，屏蔽了不同底层硬件的差异，简化了多流同步的代码实现，避免了后续新硬件接入对执行器的侵入式改动，从而进一步提高了可扩展性和可维护性。

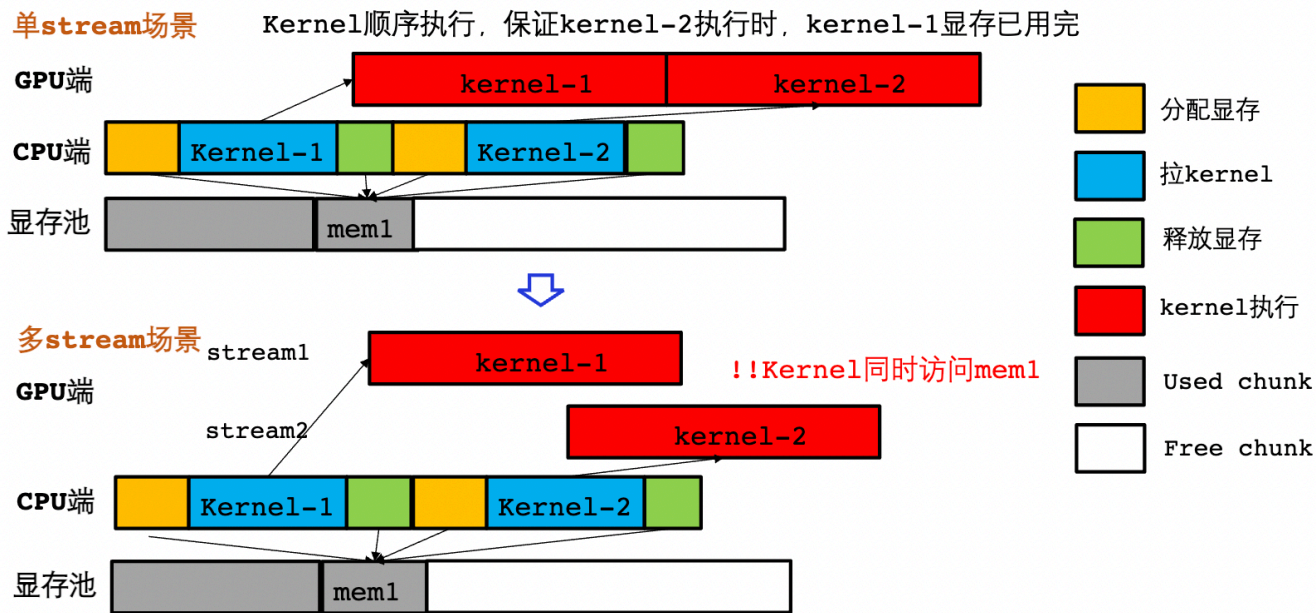


3. 显存GC

飞桨框架主要的显存优化机制是GC，并且为了最大限度地提升显存利用率，旧的执行器在单stream场景中普遍使用了Fast GC技术。

Fast GC技术可以在GPU kernel未实际跑完时就把需要用到的显存提前回收到显存缓存池中。如下图所示，在kernel-1被launch之后立刻回收mem1的显存，并且分配给kernel-2；此时，kernel-1在GPU上还未开始执行或者还未执行完，但是因为kernel-1和kernel-2在同一个stream中执行，所以他们访问mem1是有序的，不会产生数据冲突。

新执行器支持多stream执行，而在多stream的情况下，Fast GC是不安全的。因为相同的场景下，kernel-1和kernel-2被launch在不同的stream上，他们执行的先后顺序是没有约束的，可能两个kernel同时访问mem1，或者kernel-2先于kernel-1访问mem1，这两种情况都不符合程序原本的逻辑，也会造成数据错误。



多stream场景下Fast GC技术的不安全性

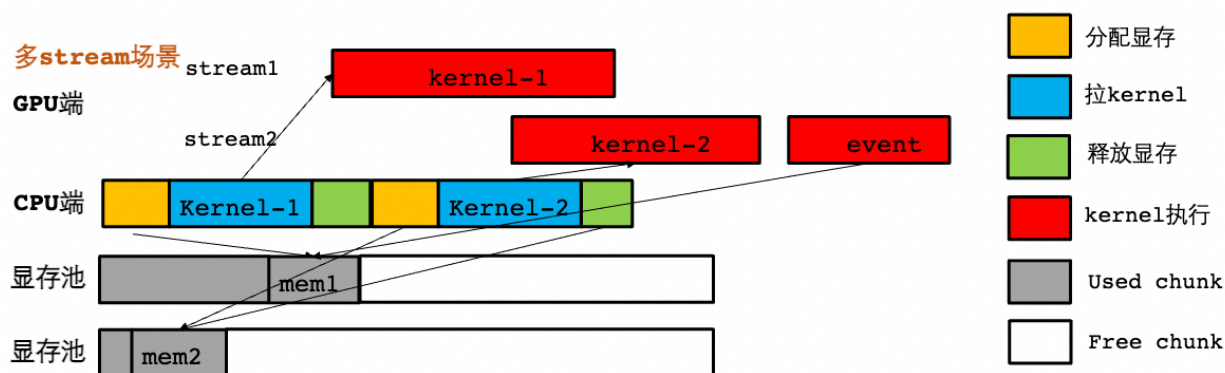
为了保证新执行器下Fast GC的正确性和高效性，新执行器结合多stream安全allocator设计了多stream场景下的fast gc。具体来说，包括以下要点：

(1) 多stream安全的allocator：不同stream隔离不同的显存池，保证释放的显存不会分配给其它stream的kernel使用

(2) Fast GC处理：

- 只被同stream使用的显存块可以提前回收
- 跨stream使用的显存块，通过event机制进行管理和同步，仅当所有stream中用到此显存块的kernel都已经执行完，才会真正回收显存，保证异步执行的安全性（使用RecordStream接口）

如下图所示，当kernel-1和kernel-2在不同stream上执行时，底层allocator对每个stream设置独立的显存池，避免混用。当一个stream中分配的显存被其他stream的kernel使用时，如下图中mem1在stream1中首次分配，然后被stream2的kernel-2使用，则会在kernel-2后面记录一个event，只有该event完成之后，mem1才会真正释放回stream1以供其它kernel复用。



kernel2同时使用到mem1和mem2，mem2在kernel拉起后就可以释放，mem1需要等kernel执行完

结合多stream安全allocator实现高效的Fast GC